

Bayesian GAMs with Turing.jl

GAM.jl Contributors

Table of contents

Introduction	2
Setup	2
Example 1: Gaussian GAM	2
Data	2
Frequentist reference	3
Bayesian fit	3
Understanding the priors	4
Posterior summaries	4
Accessing posterior samples	4
Posterior density of <code>_obs</code>	5
Comparing posteriors: frequentist vs Bayesian	5
Posterior density of intercept [1]	6
Empirical CDF comparison	6
Posterior predictive check	7
Example 2: Poisson GAM	7
Data	7
Frequentist vs Bayesian	7
Posterior summary	7
ECDF comparison for intercept	8
Cross-language comparison: GAM.jl vs brms vs mgcv	8
ECDF correlation helper	8
Gaussian model	8
ECDF comparison plots (Gaussian)	9
Poisson model	10
Example 3: Custom priors — effect on smoothing	10
Prior sensitivity: <code>_s</code> posterior	11
Example 4: Building custom Turing models	11
Summary	13
Key design choices	13

Introduction

A **Bayesian GAM** replaces the penalized likelihood framework of a standard GAM with a fully probabilistic model. Instead of choosing smoothing parameters by REML or GCV, the Bayesian approach places **priors** on the smooth function variability and uses **MCMC** (Markov chain Monte Carlo) to obtain posterior distributions for all parameters.

The connection between penalized splines and Bayesian models is well established:

- A penalized spline $f(x) = \mathbf{Z}\mathbf{b}$ with penalty $\lambda\|\mathbf{b}\|^2$ is equivalent to a random effect $\mathbf{b} \sim N(0, \sigma_s^2 \mathbf{I})$ where $\sigma_s^2 = \sigma^2/\lambda$
- The **smooth2random** decomposition splits each smooth into a fixed null-space part (unpenalized) and a random-effects part (penalized)
- Priors on σ_s control the amount of smoothing — this is the Bayesian analog of REML estimation of λ

GAM.jl implements this via a **Turing.jl package extension**: simply pass `priors = PriorSpec(...)` to `gam()` and the model is automatically reparameterized and sampled using NUTS (No-U-Turn Sampler).

Setup

```
using GAM
using Turing
using CSV
using DataFrames
using Distributions
using Statistics: mean, std, var, cor, median, quantile
using StatsAPI: coef, coeftable, confint, fitted, nobs
using Plots
using Printf
using LinearAlgebra: I
```

Example 1: Gaussian GAM

Data

Simulated data with $y = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim N(0, 0.3^2)$.

`n = 200`, `y range: [-1.48, 1.61]`

Frequentist reference

```
m_freq = gam(@gam_formula(y ~ s(x, k = 10)), dat)
freq_int = coef(m_freq)[1]
freq_ = sqrt(m_freq.scale)
```

0.3133626090202822

Frequentist: intercept = -0.1129, = 0.3134, edf = 7.8

Bayesian fit

To trigger Bayesian fitting, pass a `PriorSpec` to `gam()`. The same formula syntax is used — the dispatch is automatic:

```
m_bayes = gam(@gam_formula(y ~ s(x, k = 10)), dat;
  priors = PriorSpec(sds = Exponential(1.0)),
  nsamples = 2000, nchains = 2)
```

Bayesian Generalized Additive Model

Formula: y ~ 1

Family: Normal

Link: IdentityLink

Sampler: Turing.Inference.NUTS{ADTypes.AutoForwardDiff{nothing, Nothing}, AdvancedHMC.DiagEu

Parametric coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI
(Intercept)	-0.113137	0.0220122	-0.155759	-0.0703543
s(x,bs=tp)_f1	-3.4783	1.05015	-5.49588	-1.44708

Smooth terms: s(x,bs=tp)

n = 200

Understanding the priors

PriorSpec controls the prior distributions:

- **sds**: Prior on σ_s , the SD of smooth random effects. An `Exponential(1.0)` prior is weakly informative, allowing the data to determine smoothness.
- **sigma**: Prior on σ_{obs} , the residual SD (Gaussian family only). Default: `truncated(Normal(0, 2.5); lower=0)`.
- **b**: Prior on fixed-effect coefficients. Default: `Normal(0, 10)`.

PriorSpec:

```
b (fixed effects):    Distributions.Normal{Float64}(=0.0, =10.0)
sds (smooth SDs):    Distributions.Exponential{Float64}(=1.0)
sigma (residual SD): Truncated(Distributions.Normal{Float64}(=0.0, =2.5); lower=0.0)
phi (dispersion):    Truncated(Distributions.Normal{Float64}(=0.0, =5.0); lower=0.0)
```

Posterior summaries

The `coefstable()` method returns posterior mean, SD, and 95% credible intervals:

	Estimate	Est.Error	l-95% CI	u-95% CI
(Intercept)	-0.113137	0.0220122	-0.155759	-0.0703543
s(x,bs=tp)_f1	-3.4783	1.05015	-5.49588	-1.44708

Credible intervals at different levels:

Intercept CIs:

```
90%: [-0.1496, -0.0768]
95%: [-0.1558, -0.0704]
```

Accessing posterior samples

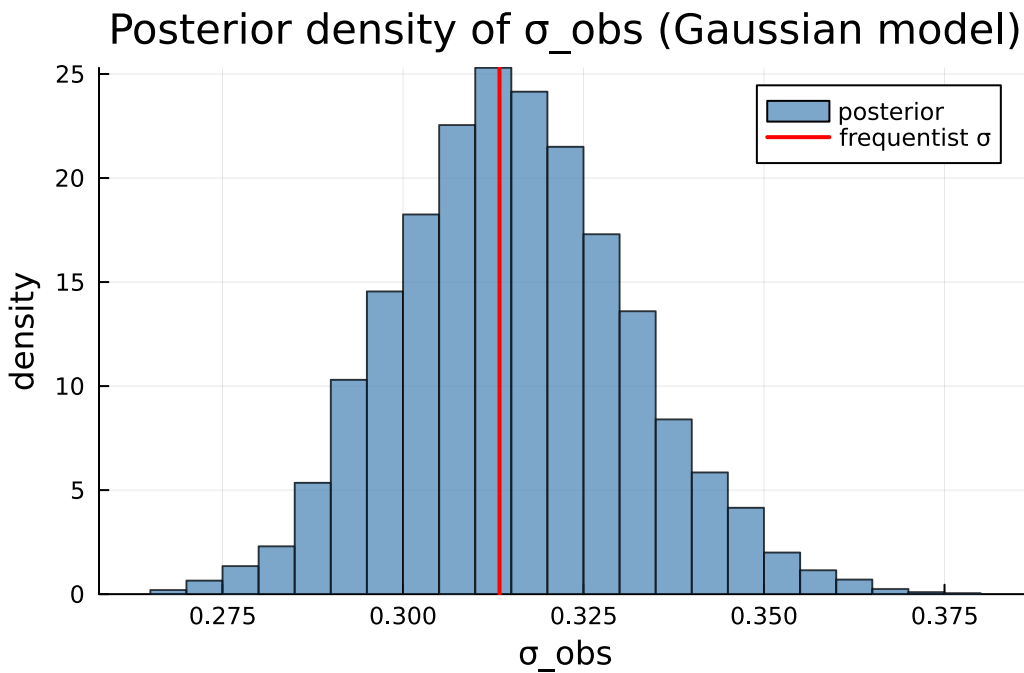
The full MCMC chains are accessible via `m_bayes.chains`:

_obs posterior: mean = 0.3154, sd = 0.0162, median = 0.3148
95% CI: [0.2855, 0.3489]
Frequentist : 0.3134

_s[1] posterior: mean = 1.6154, sd = 0.4265
Larger _s → more flexible smooth; smaller → smoother

Posterior density of σ_{obs}

```
histogram(_obs; normalize=:pdf, bins=50, alpha=0.7, color=:steelblue,  
  label="posterior", xlabel="_obs", ylabel="density",  
  title="Posterior density of _obs (Gaussian model)")  
vline!([freq_]; color=:red, linewidth=2, label="frequentist ")
```

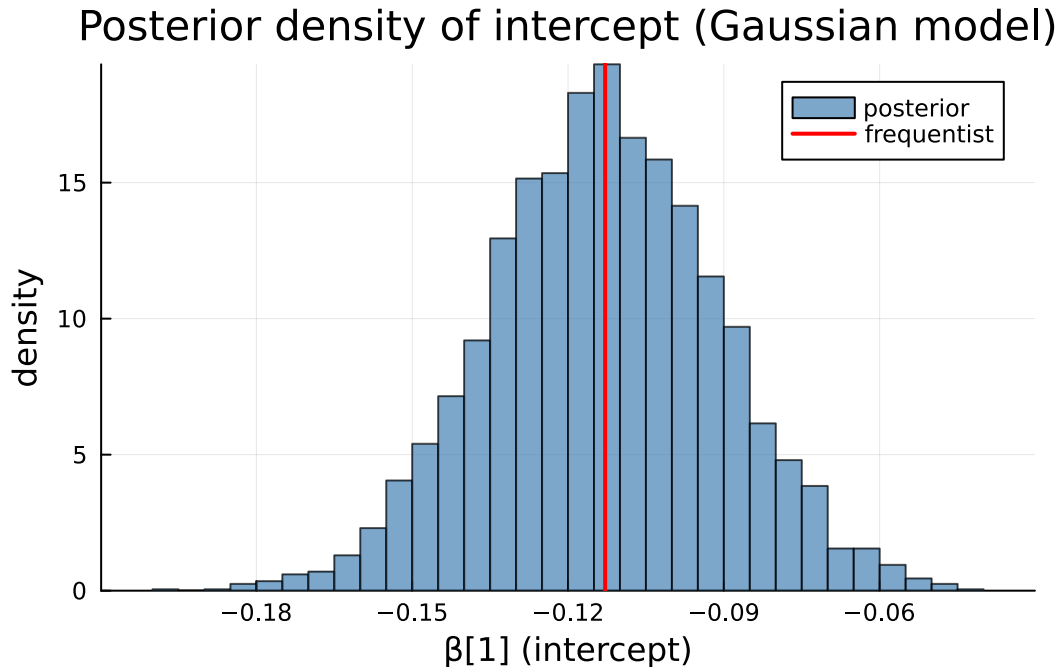


Comparing posteriors: frequentist vs Bayesian

Intercept: frequentist = -0.1129, Bayesian posterior mean = -0.1131
: frequentist = 0.3134, Bayesian posterior mean = 0.3154

Posterior density of intercept [1]

```
1_samples = vec(chains[Symbol(" [1]")].data)
histogram(1_samples; normalize=:pdf, bins=50, alpha=0.7, color=:steelblue,
  label="posterior", xlabel=" [1] (intercept)", ylabel="density",
  title="Posterior density of intercept (Gaussian model)")
vline!([freq_int]; color=:red, linewidth=2, label="frequentist")
```



Empirical CDF comparison

We compare the posterior distribution of σ_{obs} against the frequentist point estimate using the empirical CDF:

```
Frequentist = 0.3134 sits at 46.3% of the posterior ECDF
(values near 50% indicate good agreement)
ECDF 2.5%: _obs = 0.2854
ECDF 25.0%: _obs = 0.3040
ECDF 50.0%: _obs = 0.3148
ECDF 75.0%: _obs = 0.3259
ECDF 97.5%: _obs = 0.3489
```

Posterior predictive check

Compare the posterior mean fitted values to the frequentist fitted values:

Number of coefficients: frequentist = 10, Bayesian = 2

Example 2: Poisson GAM

Data

Count data with $\log(\lambda) = 1 + 1.5 \sin(2\pi x)$.

n = 200, y range: [0, 17]

Frequentist vs Bayesian

```
m_freq2 = gam(@gam_formula(y ~ s(x, k = 10)), dat2;
              family = Poisson(), link = LogLink())

m_bayes2 = gam(@gam_formula(y ~ s(x, k = 10)), dat2;
              family = Poisson(), link = LogLink(),
              priors = PriorSpec(sds = Exponential(1.0)),
              nsamples = 2000, nchains = 2)
```

Posterior summary

	Estimate	Est.Error	l-95% CI	u-95% CI
(Intercept)	0.907184	0.0542721	0.79366	1.01152
s(x,bs=tp)_f1	-3.63006	1.65901	-6.85482	-0.355444

Intercept (log-scale): frequentist = 0.9094, Bayesian = 0.9072 (true = 1.0)
_s[1]: mean = 2.2228, sd = 0.5971

ECDF comparison for intercept

Frequentist intercept = 0.9094 sits at 50.5% of Bayesian posterior

True intercept = 1.0 sits at 96.0% of posterior

Cross-language comparison: GAM.jl vs brms vs mgcv

We load posterior samples from R's brms (Stan MCMC) and mgcv (Gaussian approximation) to compare with GAM.jl's Turing posteriors. The key metric is the **pairwise ECDF correlation**: evaluate both ECDFs on a common grid and compute Pearson's r . A value of 1.0 means the distributions are identical.

Loaded: brms Gaussian=2000, brms Poisson=2000, mgcv Gaussian=4000, mgcv Poisson=4000

ECDF correlation helper

```
function ecdf_cor(x::AbstractVector, y::AbstractVector; n_grid::Int = 500)
    lo = min(minimum(x), minimum(y))
    hi = max(maximum(x), maximum(y))
    grid = range(lo, hi; length = n_grid)
    ecdf_x = [count((g), x) / length(x) for g in grid]
    ecdf_y = [count((g), y) / length(y) for g in grid]
    return cor(ecdf_x, ecdf_y)
end
```

ecdf_cor (generic function with 1 method)

Gaussian model

Parameter	GAM.jl (Turing)	brms (Stan)	mgcv (approx)
sigma mean	0.3154	0.3151	0.3145
sigma sd	0.0162	0.0161	0.0160
Intercept mean	-0.1131	-0.1131	-0.1123
Intercept sd	0.0220	0.0223	0.0221

Pairwise ECDF correlation (Gaussian model):

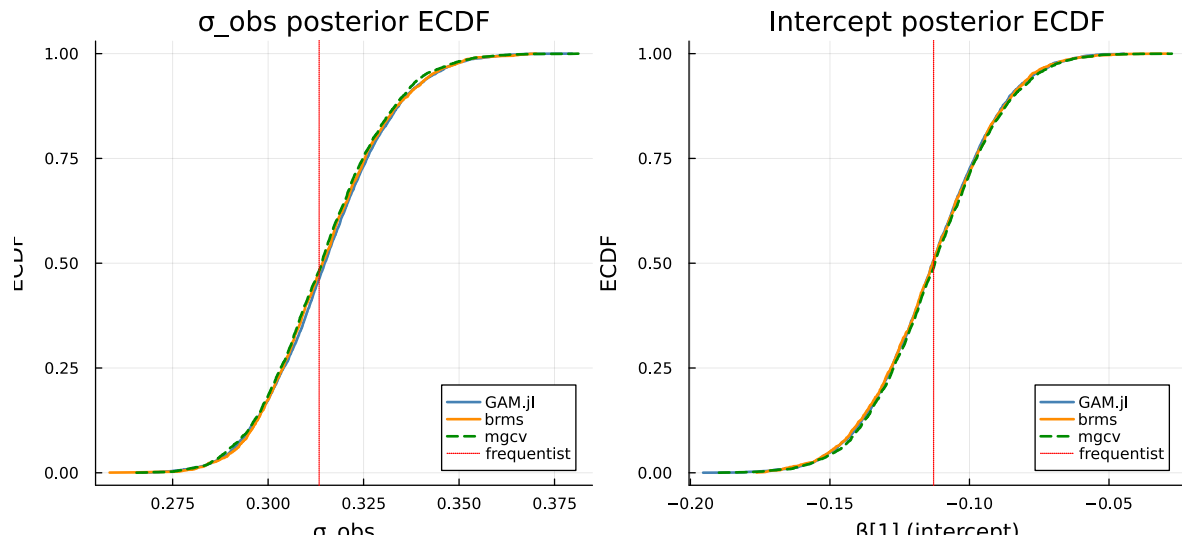
Comparison	sigma ECDF cor	intercept ECDF cor
----- ----- -----		
GAM.jl vs brms	0.999870	0.999970
GAM.jl vs mgcv (approx)	0.999731	0.999927
brms vs mgcv (approx)	0.999921	0.999904

ECDF comparison plots (Gaussian)

```
p1 = plot(sort(julia_sigma), (1:length(julia_sigma)) ./ length(julia_sigma);
        label="GAM.jl", color=:steelblue, linewidth=2,
        xlabel="_obs", ylabel="ECDF", title="_obs posterior ECDF")
plot!(p1, sort(brms_sigma), (1:length(brms_sigma)) ./ length(brms_sigma);
        label="brms", color=:darkorange, linewidth=2)
plot!(p1, sort(mgcv_sigma), (1:length(mgcv_sigma)) ./ length(mgcv_sigma);
        label="mgcv", color=:green4, linewidth=2, linestyle=:dash)
vline!(p1, [freq_]; color=:red, linewidth=1, linestyle=:dot, label="frequentist")

p2 = plot(sort(julia_int), (1:length(julia_int)) ./ length(julia_int);
        label="GAM.jl", color=:steelblue, linewidth=2,
        xlabel="[1] (intercept)", ylabel="ECDF", title="Intercept posterior ECDF")
plot!(p2, sort(brms_int), (1:length(brms_int)) ./ length(brms_int);
        label="brms", color=:darkorange, linewidth=2)
plot!(p2, sort(mgcv_int), (1:length(mgcv_int)) ./ length(mgcv_int);
        label="mgcv", color=:green4, linewidth=2, linestyle=:dash)
vline!(p2, [freq_int]; color=:red, linewidth=1, linestyle=:dot, label="frequentist")

plot(p1, p2; layout=(1, 2), size=(900, 400), legend=:bottomright)
```



Poisson model

Pairwise ECDF correlation (Poisson intercept):

GAM.jl vs brms: 0.999920

GAM.jl vs mgcv (approx): 0.999937

brms vs mgcv (approx): 0.999957

Example 3: Custom priors — effect on smoothing

The prior on `sds` directly controls the amount of smoothing. A tighter prior yields smoother fits:

```
# Tight prior: Exponential(0.1) → small _s → smoother
m_tight = gam(@gam_formula(y ~ s(x, k = 10)), dat;
  priors = PriorSpec(sds = Exponential(0.1)),
  nsamples = 1000, nchains = 1)

# Wide prior: Exponential(5.0) → large _s → wigglier
m_wide = gam(@gam_formula(y ~ s(x, k = 10)), dat;
  priors = PriorSpec(sds = Exponential(5.0)),
  nsamples = 1000, nchains = 1)
```

Tight prior (Exp(0.1)): posterior mean `_s` = 0.9995

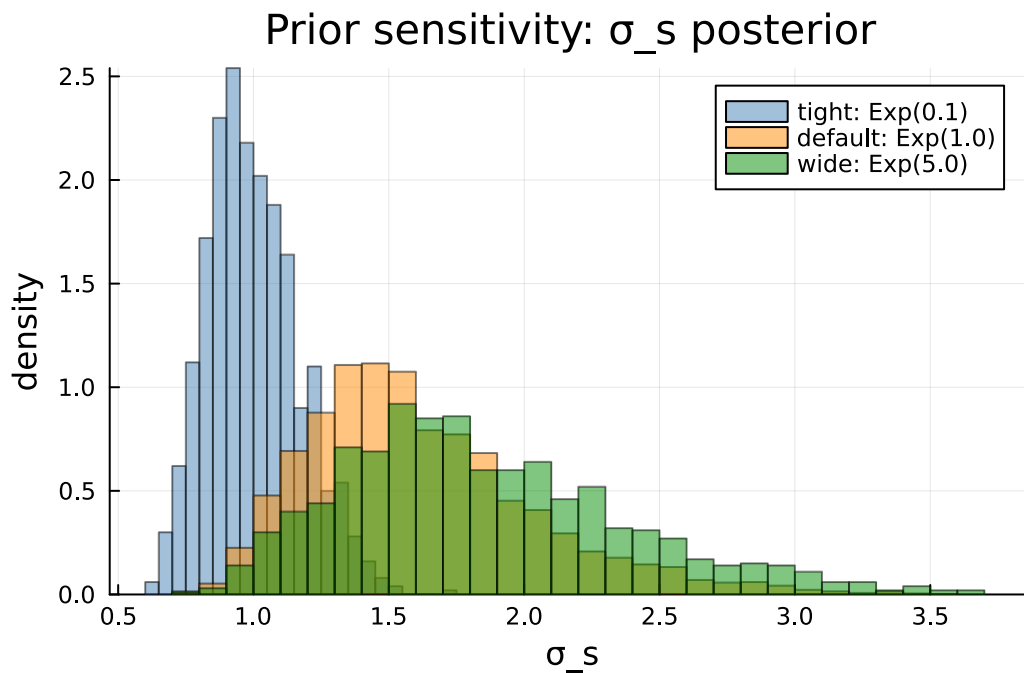
Wide prior (Exp(5.0)): posterior mean `_s` = 1.8479

Ratio: 1.8x

Prior sensitivity: σ_s posterior

```
_s_tight = vec(m_tight.chains[Symbol("_s[1]")].data)
_s_default = _s # from default Exp(1.0) model
_s_wide = vec(m_wide.chains[Symbol("_s[1]")].data)

histogram(_s_tight; normalize=:pdf, bins=30, alpha=0.5, color=:steelblue,
  label="tight: Exp(0.1)", xlabel="_s", ylabel="density",
  title="Prior sensitivity:  $\sigma_s$  posterior")
histogram!(_s_default; normalize=:pdf, bins=30, alpha=0.5, color=:darkorange,
  label="default: Exp(1.0)")
histogram!(_s_wide; normalize=:pdf, bins=30, alpha=0.5, color=:green4,
  label="wide: Exp(5.0)")
```



Example 4: Building custom Turing models

For full control, use `gam_matrices()` and `gam_smooth()` to extract the basis matrices, then write your own `@model`:

```
# Extract matrices from a formula
gf = @gam_formula(y ~ s(x, k = 10))
X, sms, labels = gam_matrices(gf, dat)
```

```
([1.0; 1.0; ... ; 1.0; 1.0;;], SmoothMixedModel[SmoothMixedModel([0.664327533630193; 0.6628743
0.540699473296364; -0.5426167225557019;;], [[0.15438623644410757 -0.2577943937355174 ... 0.035
0.7659102718826961; 0.15440736781494313 -0.25766668968864265 ... 0.03788466790460092 -
0.7616640417613573; ... ; -0.13612135859607952 0.1606164144848799 ... -0.6484895009581195 -
0.03752349474514074; -0.13611571860612945 0.16045612470122486 ... -0.6500996527498536 -
0.04057732998841621]], [0.014406173358386995 -0.026837067992358504 ... -0.05154417204031194 6.
16; 0.04344154074823492 0.004410031457064265 ... 0.5600267625508373 -5.399717439754989e-
15; ... ; 0.05008586618744659 0.012165648317958355 ... 0.5322891433528174 0.6649355842620508; 0.
0.746900708784029], [0.10378107657020536, 0.11887176712494019, 0.17586602174257748, 0.216874
```

```
Fixed matrix X: (200, 1) (intercept)
Smooth 's(x,bs=tp)':
  Xf (null space): (200, 1)
  Zs (penalized): [(200, 8)]
```

```
# Build custom model
Xf = sms[1].Xf
Zs = sms[1].Zs[1]
X_fixed = hcat(X, Xf)
```

```
@model function my_gam(y_obs, X_f, Z)
  n_f = size(X_f, 2)
  n_z = size(Z, 2)
```

```
  # Priors
  ~ MvNormal(zeros(n_f), 10.0 * I)
  ~ truncated(Normal(0, 2.5); lower = 0.0)
  _s ~ Exponential(1.0)
```

```
  # Random effects (non-centered parameterization)
  z ~ MvNormal(zeros(n_z), I)
  = X_f * . + _s .* (Z * z)
```

```
  # Likelihood
  y_obs ~ MvNormal(, ^2 * I)
```

```
end
```

```
custom_chains = sample(my_gam(dat.y, X_fixed, Zs), NUTS(), 1000; progress = false)
```

```
Info: Found initial step size
      = 0.2
```

```
Chains MCMC chain (1000×26×1 Array{Float64, 3}):
```

```
Iterations          = 501:1:1500
Number of chains    = 1
Samples per chain   = 1000
Wall duration       = 5.79 seconds
Compute duration    = 5.79 seconds
parameters          = [1], [2], , _s, z[1], z[2], z[3], z[4], z[5], z[6], z[7], z[8]
internals           = n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, ha
```

Use ``describe(chains)`` for summary statistics and quantiles.

Custom model posterior mean: 0.3146 (compare to 0.3154 from `gam()`)

Summary

Feature	Syntax
Default Bayesian GAM	<code>gam(formula, data; priors = PriorSpec())</code>
Custom priors	<code>PriorSpec(sds = Exponential(1.0), sigma = InverseGamma(2, 3))</code>
Per-smooth priors	<code>PriorSpec(specific = Dict("sds_s(x2)" => Exponential(0.5)))</code>
Poisson	<code>gam(formula, data; family = Poisson(), priors = PriorSpec())</code>
Coefficient table	<code>coefstable(m)</code> — posterior mean, SD, 95% CI
Credible intervals	<code>confint(m; level = 0.95)</code>
Full posterior	<code>m.chains[Symbol("_obs")]</code>
Custom model	<code>gam_matrices() + @model + sample()</code>

Key design choices

1. **Dispatch-based API:** The same `gam()` function handles both frequentist and Bayesian fitting — the presence of `priors=` triggers Bayesian mode via multiple dispatch.

2. **smooth2random decomposition:** Smooth terms are split into a fixed null-space (unpenalized, estimated via `<math>\text{glm}</math>`) and random-effects blocks (penalized, estimated via `<math>\text{glmer}</math>`).
3. **Non-centered parameterization:** Random effects use $\mathbf{b} = \sigma_s \cdot \mathbf{z}$ where $\mathbf{z} \sim N(0, I)$, which improves NUTS sampling geometry.
4. **Package extension:** Turing.jl is only loaded when needed (`using Turing`), keeping the base GAM.jl lightweight.